Ministère de l'Enseignement
Supérieur et de la Recherche
Scientifique

Année Universitaire
2026/2027

EPI DIGITAL SCHOOL
Ecole Internationale Supérieure
Privée Polytechnique de Sousse

Rapport de stage professionnel

Spécialité: .....

Intitulé

.....

Lieu du stage
SAFYRON ENGINEERING
Tunis

Réalisé par

Encadré par
M. Achref Ammar

Remerciements

Je tiens à remercier M. Achref Ammar pour son accueil, sa disponibilité, son encadrement professionnel et la confiance qu'il m'a accordée pendant ce stage. Son accompagnement m'a permis de mieux comprendre le contexte de l'entreprise et d'inscrire mon travail dans une démarche structurée.

J'adresse également mes sincères remerciements à SAFYRON ENGINEERING pour l'opportunité de découvrir son environnement professionnel et les problématiques liées au traitement et au suivi des demandes techniques.

Enfin, je remercie EPI Digital School Sousse pour la formation en génie logiciel et pour le cadre académique du stage professionnel, qui m'ont permis de mettre en pratique les connaissances acquises et de conduire une démarche complète, de l'analyse du besoin jusqu'à la validation de la solution.

Résumé

Ce rapport présente un projet de conception et de réalisation d'une application web interne dédiée à la gestion des demandes techniques, inscrit dans le cadre d'un stage professionnel d'un mois. Le projet couvre l'enregistrement, la qualification, l'affectation, le traitement, la résolution et la clôture des demandes, avec une historisation des changements importants. L'architecture associe un frontend Angular, une API REST Spring Boot et une base PostgreSQL, selon une organisation modulaire séparant présentation, application, domaine et persistance. La sécurité repose sur l'authentification JWT, le contrôle d'accès fondé sur les rôles et des contrôles métier contextuels. Une assistance IA intégrée via Groq propose une catégorie et une priorité à partir du titre et de la description ; le Responsable technique conserve la décision finale. La validation s'appuie sur des tests automatisés backend et frontend, une intégration locale du parcours IA et une vérification fonctionnelle humaine. Le rapport distingue les résultats mesurés sur le périmètre retenu pour le stage des limites, notamment le caractère exploratoire du benchmark IA et l'absence de portail client.

Mots-clés : génie logiciel ; API REST ; Angular ; Spring Boot ; intelligence artificielle.

Abstract

This report presents a project to design and implement an internal web application for managing technical requests, undertaken in the context of a one-month professional internship. The solution addresses a controlled internal scope. The project covers request registration, qualification, assignment, processing, resolution and closure, with the history of significant changes. The architecture combines an Angular frontend, a Spring Boot REST API and a PostgreSQL database, using modular responsibilities for presentation, application, domain and persistence. Security relies on JWT authentication, role-based access control and contextual business checks. An AI assistance feature integrated through Groq proposes a category and a priority from the request title and description ; the Technical Manager remains responsible for the final decision. Validation combines automated backend and frontend tests, a local integration of the AI workflow and human functional verification. The report distinguishes measured results within the scope retained for the internship from remaining limitations, including the exploratory nature of the AI benchmark and the absence of a customer portal.

Keywords : software engineering ; REST API ; Angular ; Spring Boot ; artificial intelligence.

Sommaire

Liste des figures **iv**

Liste des tableaux **v**

Liste des acronymes **vi**

Introduction générale **1**

Chapitre 1 — Contexte et cadrage du projet **2**
1.1 Organisme d'accueil et cadre du stage 2
1.2 Contexte, existant et problématique 3
1.3 Solution proposée, objectifs et périmètre 3
1.4 Méthodologie et planification du travail 4

Chapitre 2 — Analyse et conception **5**
Introduction 5
2.1 Spécification des besoins 5
2.1.1 Acteurs et responsabilités 5
2.1.2 Besoins fonctionnels 5
2.1.3 Besoins non fonctionnels 6
2.2 Conception fonctionnelle 6
2.3 Conception technique 9
2.3.1 Architecture générale 9
2.3.2 Principes de conception et patterns 9
2.3.3 Sécurité de la solution 12
2.3.4 Assistance intelligente 12
2.4 Conception de la base de données 12
Conclusion 14

Chapitre 3 — Réalisation et validation **15**
Introduction 15
3.1 Environnement de réalisation et technologies utilisées 15
3.2 Réalisation des fonctionnalités principales 16
3.3 Administration et sécurité 20
3.4 Assistance IA et validation humaine 21
3.5 Tests et validation 22
3.6 Difficultés rencontrées, limites et perspectives 24
Conclusion 25

Conclusion générale **26**

Références **27**

Liste des figures

1 Logo de SAFYRON ENGINEERING 2

2 Diagramme global des cas d'utilisation 7
3 Enregistrement et qualification des demandes 8
4 Composants applicatifs et systèmes externes 9
5 Organisation des responsabilités dans la conception backend 11
6 Entités persistantes et cardinalités du modèle de données 14

7 Formulaire de création et de qualification d'une demande 17
8 Suivi des demandes avec filtres, tri et pagination 18
9 Traitement d'une demande affectée 19
10 Clôture après examen de la résolution 20
11 Gestion des utilisateurs internes 21
12 Assistance IA lors de la qualification 22

Liste des tableaux

1 Diagramme de Gantt simplifié — planning prévisionnel du stage 4

2 Capacités fonctionnelles de la solution 6
3 Dictionnaire de données synthétique 13

4 Technologies et versions du projet 16
5 Résultats de validation par niveau 23
6 Traçabilité des besoins non fonctionnels et des validations 24

Liste des acronymes

API	Interface de programmation d'application (*Application Programming Interface*).
DTO	Objet de transfert de données (*Data Transfer Object*).
HTTP	Protocole de transfert hypertexte (*Hypertext Transfer Protocol*).
IA	Intelligence artificielle.
JPA	Interface de persistance Java (*Java Persistence API*).
JWT	Jeton web JSON (*JSON Web Token*).
RBAC	Contrôle d'accès fondé sur les rôles (*Role-Based Access Control*).
REST	Transfert d'état représentationnel (*Representational State Transfer*).
UML	Langage de modélisation unifié (*Unified Modeling Language*).

Introduction générale

Le stage professionnel s'est déroulé du 1er au 31 août 2026 au sein de SAFYRON ENGINEERING. Cette période d'un mois a été consacrée à un projet de génie logiciel lié au suivi des demandes techniques formulées par les clients de l'entreprise.

Dans les activités de l'entreprise, les demandes techniques formulées par les clients doivent être comprises, qualifiées et suivies jusqu'à l'achèvement de leur traitement. Chaque demande rassemble plusieurs informations utiles, notamment le client concerné, la nature du besoin, son niveau de priorité, la personne chargée du traitement et son état d'avancement. La continuité de ce suivi suppose que ces informations restent accessibles et cohérentes au fil des différentes étapes.

Le besoin général consiste ainsi à disposer d'un moyen commun pour centraliser les demandes, faciliter leur qualification, attribuer les responsabilités et conserver une trace de leur évolution. Il ne s'agit pas uniquement d'enregistrer une nouvelle demande, mais d'accompagner l'ensemble de son cycle de vie tout en maintenant un accès adapté aux responsabilités des utilisateurs internes.

La problématique retenue est donc la suivante : comment concevoir une solution interne cohérente permettant de centraliser, qualifier, affecter et suivre les demandes techniques des clients, tout en assurant la traçabilité de leur évolution et le contrôle humain des décisions importantes ?

Pour répondre à cette problématique, le projet propose une application web interne dédiée à la gestion des demandes techniques. Son objectif est de fournir un espace structuré pour leur prise en charge, depuis l'enregistrement jusqu'à la clôture. Une assistance fondée sur l'intelligence artificielle peut compléter la qualification, mais elle demeure facultative et ne remplace jamais la décision de l'utilisateur responsable.

Le rapport est organisé en trois chapitres. Le premier présente l'organisme d'accueil, le contexte, la problématique, les objectifs et la planification du stage. Le deuxième expose l'analyse des besoins et la conception de la solution. Le troisième décrit la réalisation, les mécanismes essentiels et les éléments de validation obtenus.

Chapitre 1

Contexte et cadrage du projet

Le cadre professionnel permet de situer l'origine du besoin et la problématique étudiée. La solution proposée, son périmètre et les objectifs retenus sont ensuite présentés, avant la démarche de travail et la planification du stage.

1.1 Organisme d'accueil et cadre du stage

Le stage professionnel s'est déroulé du 1er au 31 août 2026 au sein de SAFYRON ENGINEERING, située rue Lac Toba, aux Berges du Lac à Tunis. L'entreprise exerce dans le domaine des études techniques industrielles. Ses activités comprennent notamment la protection incendie, les études de dangers et l'analyse des risques industriels.

La figure 1 présente l'identité visuelle de l'organisme d'accueil.



FIGURE 1 – Logo de SAFYRON ENGINEERING

Le stage professionnel d'un mois a été réalisé dans le cadre de la cinquième année du cycle d'ingénieur en génie logiciel à EPI Digital School Sousse. Le travail a été mené individuellement, conformément aux exigences de l'école, sans rattachement à un service particulier. L'encadrement professionnel a été assuré par M. Achref Ammar, fondateur de SAFYRON ENGINEERING.

L'observation du contexte de l'entreprise a permis de mieux comprendre la nature des études réalisées et la place des demandes formulées par les clients. Sur la base de cette compréhension des processus de travail, l'étudiant a proposé l'idée d'une application web interne de gestion et de suivi des demandes techniques.

1.2 Contexte, existant et problématique

Les activités de SAFYRON ENGINEERING conduisent à traiter des demandes techniques provenant de clients. Une demande doit être comprise, orientée vers la personne concernée et suivie jusqu'à l'achèvement de son traitement.

L'étude de l'existant porte sur la logique générale de traitement des demandes, sans présumer des outils utilisés auparavant. À partir de cette compréhension du fonctionnement métier, plusieurs informations ont été retenues comme nécessaires au suivi dans l'application : l'identité du client, la nature du besoin, le niveau de priorité, la personne chargée du traitement, l'état courant et les changements successifs. Ces informations doivent rester cohérentes au cours de la prise en charge d'une demande.

Le besoin ne consiste donc pas seulement à enregistrer une demande. Le suivi doit permettre de la retrouver, de connaître son niveau d'avancement, d'identifier la personne responsable de son traitement et de conserver la trace des décisions importantes. Les intervenants disposent ainsi d'une vue claire de l'état de chaque demande et de la responsabilité associée à sa prise en charge.

La problématique du projet peut ainsi être formulée de la manière suivante : comment mettre à la disposition des intervenants internes un moyen cohérent pour centraliser, qualifier, affecter et suivre les demandes techniques des clients, tout en assurant la traçabilité de leur évolution et un accès adapté aux responsabilités de chacun ?

1.3 Solution proposée, objectifs et périmètre

Pour répondre à cette problématique, la solution proposée est une application web interne consacrée à la gestion et au suivi des demandes techniques. Elle fournit un espace commun dans lequel les demandes peuvent être enregistrées, qualifiées, affectées et suivies pendant leur traitement.

Les objectifs principaux sont de centraliser les demandes, de faciliter leur qualification et de formaliser leur cycle de vie. La solution doit également permettre d'affecter chaque demande, de suivre son traitement, de conserver un historique des changements et de protéger l'accès selon les responsabilités des utilisateurs. Une assistance fondée sur l'intelligence artificielle peut proposer une catégorie et une priorité ; elle reste facultative et toute proposition doit être validée par une personne avant son utilisation.

Les responsabilités internes se répartissent entre trois profils. Le Responsable technique pilote la prise en charge des demandes, l'Agent technique réalise leur traitement et l'Administrateur gère les comptes internes. Leurs interactions et leurs besoins sont détaillés dans l'analyse fonctionnelle.

Le périmètre retenu pour le stage concerne les fonctions effectivement réalisées pour les utilisateurs internes. Le client y est représenté comme une entité métier associée à ses demandes,

mais il ne dispose ni d'un compte ni d'un portail. L'assistance par intelligence artificielle ne décide ni de la qualification ni de l'affectation : les choix finaux de catégorie et de priorité ainsi que les décisions de traitement restent sous contrôle humain. Ces limites correspondent à la durée du stage et distinguent la solution réalisée des évolutions possibles.

1.4 Méthodologie et planification du travail

La méthodologie retenue repose sur une démarche progressive de génie logiciel : analyse du besoin et définition du périmètre, modélisation fonctionnelle, conception, réalisation, tests et validation. Cet enchaînement relie chaque étape aux besoins identifiés et permet de contrôler progressivement la cohérence du projet.

La durée d'un mois impose de privilégier la simplicité, une progression contrôlée et des validations régulières. Le périmètre du projet a donc été limité à un ensemble cohérent de fonctions pouvant être analysées, réalisées et vérifiées pendant le stage. Le tableau 1 présente le planning prévisionnel retenu et sa répartition prévisionnelle en quatre périodes.

Tableau 1 – Diagramme de Gantt simplifié — planning prévisionnel du stage

Activité proposée	**S1** 01–07	**S2** 08–14	**S3** 15–21	**S4** 22–31
Compréhension du contexte de l'entreprise	■			
Analyse du besoin et définition du périmètre	■			
Analyse fonctionnelle et UML		■		
Conception de l'architecture et des données		■		
Réalisation backend et frontend			■	
Intégration progressive et finalisation			■	■
Tests et validation				■
Intégration de l'assistance IA et corrections finales				■

Le planning prévoit de consacrer la première semaine au contexte et au cadrage, la deuxième à l'analyse et à la conception, et la troisième principalement à la réalisation et au début de l'intégration. La dernière semaine est prévue pour finaliser la solution, conduire les vérifications et intégrer l'assistance IA ainsi que les corrections finales.

Cette répartition prévisionnelle organise les activités du stage sur quatre périodes au cours du mois.

Le périmètre retenu structure le traitement et le suivi des demandes techniques tout en laissant les décisions importantes aux utilisateurs concernés. Le chapitre suivant présente l'analyse des besoins ainsi que la conception fonctionnelle et technique.

Chapitre 2

Analyse et conception

Introduction

La conception transforme les besoins du stage en responsabilités, règles et structures cohérentes. Elle part des utilisateurs internes et du cycle de vie d'une demande, puis établit l'architecture, les mécanismes de sécurité, l'intégration de l'assistance IA et le modèle de données. Chaque choix est ainsi relié au problème qu'il doit résoudre.

2.1 Spécification des besoins

Cette section précise les personnes concernées par la solution et les attentes auxquelles elle doit répondre. Elle distingue les fonctions attendues des qualités générales du système, avant d'aborder les choix de conception.

2.1.1 Acteurs et responsabilités

L'application distingue trois rôles internes, qu'une même personne peut cumuler lorsque l'organisation l'exige. Le Client reste extérieur à cette répartition des accès.

- **Responsable technique :** il pilote la demande depuis son enregistrement et sa qualification jusqu'à la décision de clôture.
- **Agent technique :** il prend en charge les demandes qui lui sont affectées et documente leur traitement puis leur résolution.
- **Administrateur :** il maintient les comptes des utilisateurs internes et leurs rôles.
- **Client :** il constitue une entité métier associée à la demande, et non un acteur utilisateur de la solution actuelle ; aucun portail ni droit d'accès client n'est introduit.

2.1.2 Besoins fonctionnels

Le tableau 2 synthétise les grandes fonctions attendues dans le périmètre du stage, depuis l'accès interne jusqu'à l'assistance à la qualification.

TABLEAU 2 – Capacités fonctionnelles de la solution

Capacité	Comportement attendu
Accès interne	Authentifier un utilisateur et orienter son accès selon ses rôles.
Gestion des demandes	Enregistrer, consulter, filtrer et suivre les demandes techniques.
Qualification	Définir une catégorie et une priorité validées par le Responsable technique.
Workflow	Affecter, traiter, résoudre, clôturer, refuser ou annuler selon l'état courant.
Administration	Gérer les comptes, l'état actif et les rôles métier autorisés.
Assistance IA	Proposer une catégorie et une priorité, sans décision automatique.

Ces besoins décrivent le résultat attendu sans anticiper les mécanismes techniques retenus pour y répondre.

2.1.3 Besoins non fonctionnels

— **Sécurité :** chaque utilisateur accède uniquement aux fonctions autorisées par ses responsabilités.
— **Cohérence :** les informations d'une demande restent compatibles avec son évolution.
— **Traçabilité :** les changements importants intervenus pendant le traitement doivent pouvoir être compris.
— **Maintenabilité :** la solution doit pouvoir évoluer sans rendre ses responsabilités difficiles à comprendre ou à modifier.
— **Ergonomie :** l'état d'une demande doit être identifiable rapidement, y compris sur différentes tailles d'écran.
— **Continuité de service IA :** l'indisponibilité du fournisseur ne doit jamais empêcher une qualification manuelle.

2.2 Conception fonctionnelle

La conception fonctionnelle décrit les interactions attendues sans anticiper leur réalisation technique. Le diagramme global situe d'abord les responsabilités ; les cas majeurs et leurs scénarios précisent ensuite le comportement attendu.

La figure 2 présente les acteurs internes et les capacités du système auxquelles ils participent.

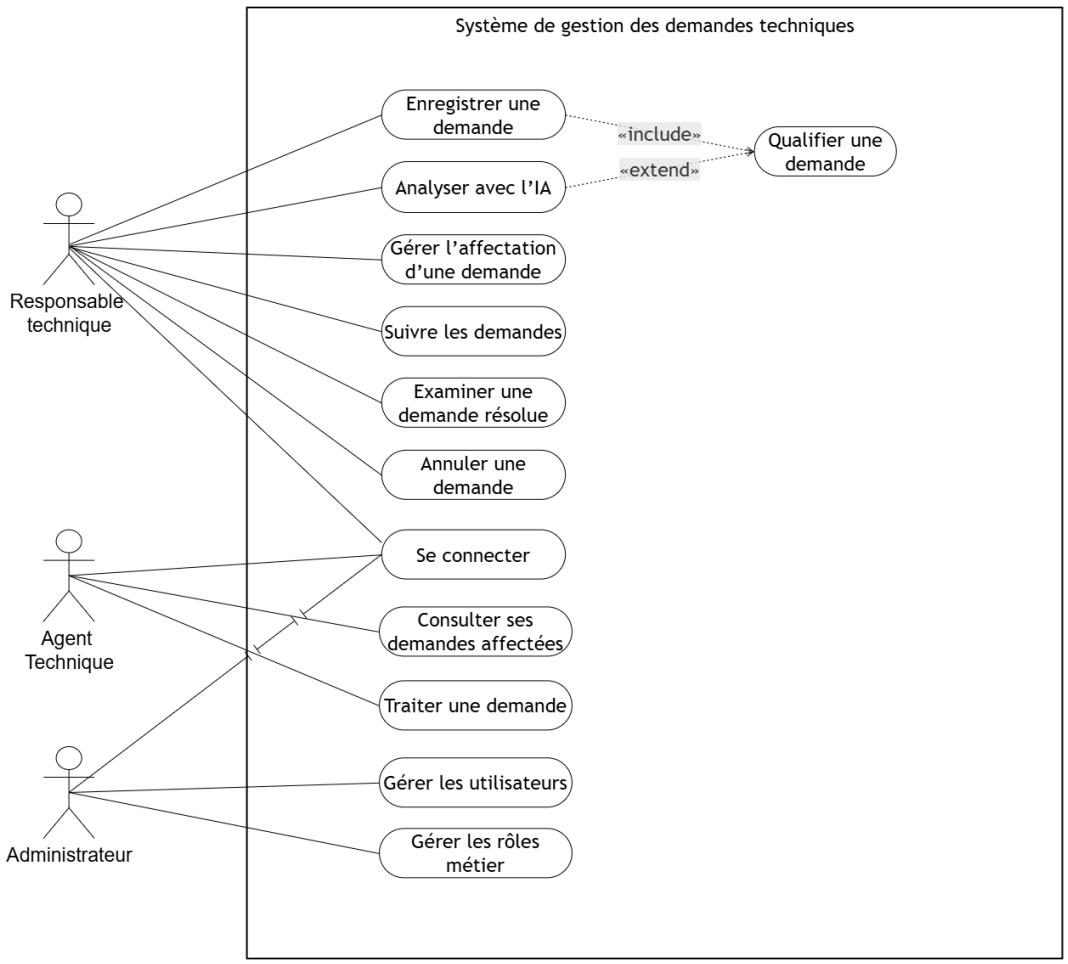


FIGURE 2 – Diagramme global des cas d'utilisation

Le diagramme distingue les responsabilités essentielles : le Responsable technique pilote le cycle de la demande, l'Agent traite les demandes qui lui sont affectées et l'Administrateur gère les utilisateurs internes. L'assistance IA reste facultative et sous le contrôle du Responsable technique.

Enregistrement et qualification. Le scénario nominal sélectionne un client existant ; l'alternative permet de saisir les informations minimales d'un nouveau client avant de valider la catégorie et la priorité.

Traitement et résolution. L'Agent documente la résolution, puis le Responsable technique l'accepte ou la refuse. Un refus replace la demande en `EN_COURS`, réinitialise la date de résolution et reste tracé.

Réaffectation et annulation. Une demande réaffectée après le début du traitement revient à `ASSIGNEE` sans perte des informations saisies. L'annulation avant clôture exige un motif.

Assistance IA. L'analyse du titre et de la description propose une catégorie et une priorité modifiables, sans décision automatique et sans bloquer la qualification manuelle.

La figure 3 détaille le scénario d'enregistrement et ses deux alternatives selon que le client existe déjà ou doit être renseigné.

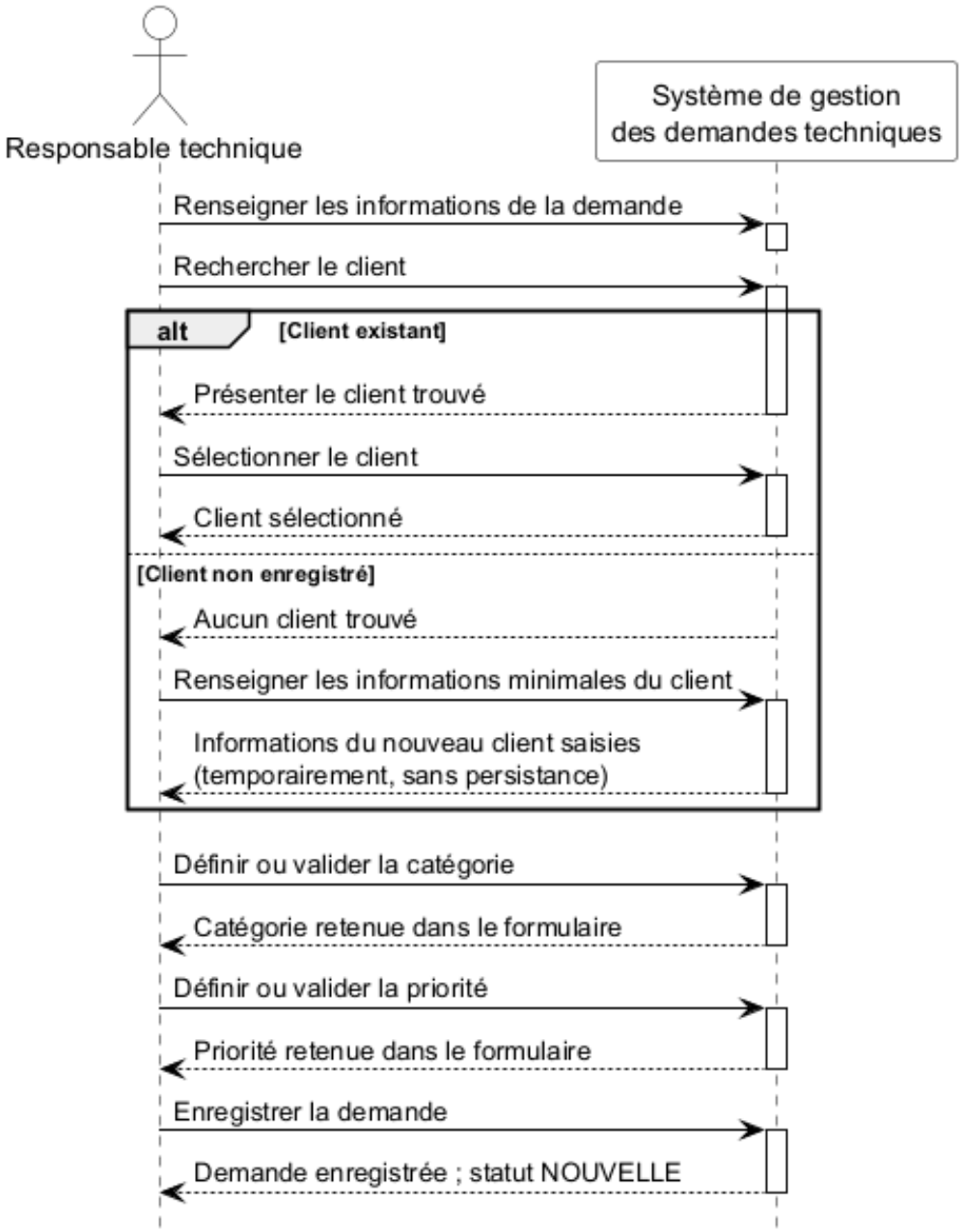


FIGURE 3 – Enregistrement et qualification des demandes

La séquence montre que le Responsable technique peut sélectionner un client existant ou renseigner un nouveau client dans le même parcours. La catégorie et la priorité sont validées avant l'enregistrement final, qui préserve la cohérence entre le client, la demande et l'événement de création.

2.3 Conception technique

Cette section traduit les besoins précédents en organisation technique. Elle présente les frontières de l'application, les principes de conception retenus, la protection des accès et l'intégration de l'assistance intelligente.

2.3.1 Architecture générale

L'architecture doit séparer l'interaction utilisateur, les règles métier, la persistance relationnelle et le fournisseur IA, tout en conservant un déploiement simple pour le périmètre retenu. La figure 4 présente les frontières retenues.

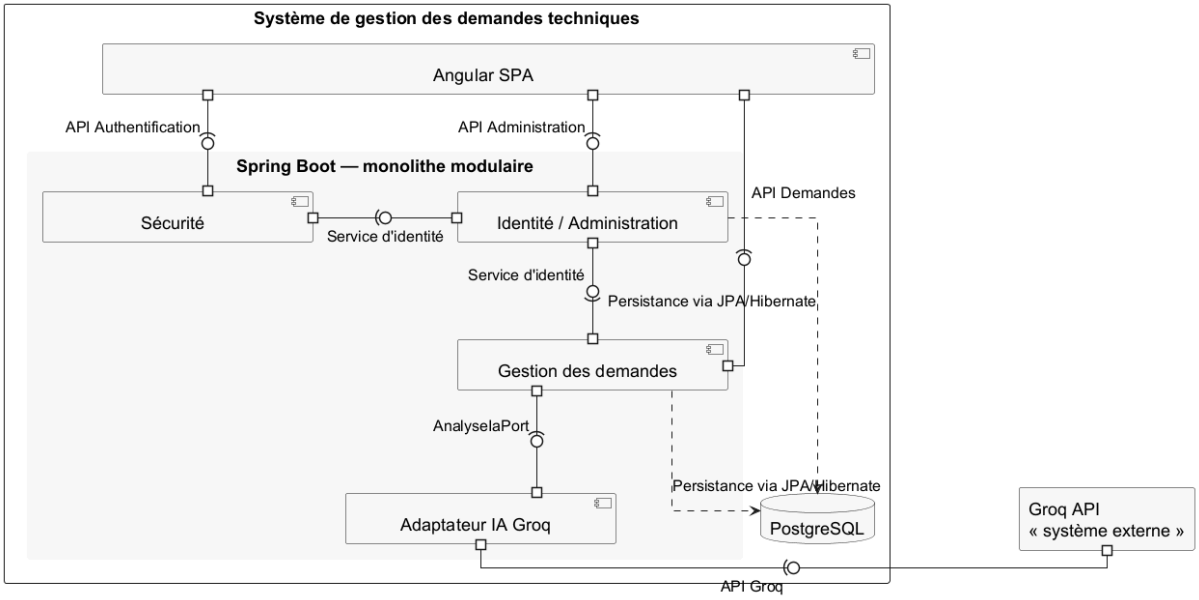


FIGURE 4 – Composants applicatifs et systèmes externes

Le diagramme sépare l'interface Angular, le backend Spring Boot et la persistance PostgreSQL. Le backend reste organisé en monolithe modulaire afin de distinguer les responsabilités sans multiplier les déploiements, tandis que l'intégration Groq est isolée du cœur métier derrière une frontière dédiée.

2.3.2 Principes de conception et patterns

Les choix retenus répondent à des besoins précis sans permettre d'affirmer que tous les principes SOLID sont appliqués uniformément.

Responsabilité unique (SRP). La séparation des rôles rend les composants plus lisibles ; contrôleurs, services, domaine et persistance gardent chacun une responsabilité principale identifiable (Martin 2017).

DIP et Port/Adapter. Le principe d'inversion des dépendances guide la séparation entre le cœur applicatif et les détails externes (Martin 2017).

`AnalyseIaPort` exprime ainsi le besoin du cœur applicatif. `GroqAnalyseIaAdapter` réalise l'appel externe selon une organisation Ports & Adapters. Cette séparation limite l'impact d'un changement de fournisseur (Cockburn 2005).

Repository. Ce pattern isole l'accès aux données des services métier (Fowler 2002).

DTO. Les objets de transfert séparent les contrats HTTP des entités persistantes, sans être présentés comme une solution universelle.

State Pattern. Il n'est pas retenu pour le périmètre actuel, car un enum de statut et des opérations métier explicites suffisent au cycle considéré. Ce choix pourrait être réévalué si le workflow s'étend fortement (Gamma et al. 1994).

La figure 5 affine la vue générale en montrant comment les responsabilités et ces choix de conception sont répartis à l'intérieur du backend.

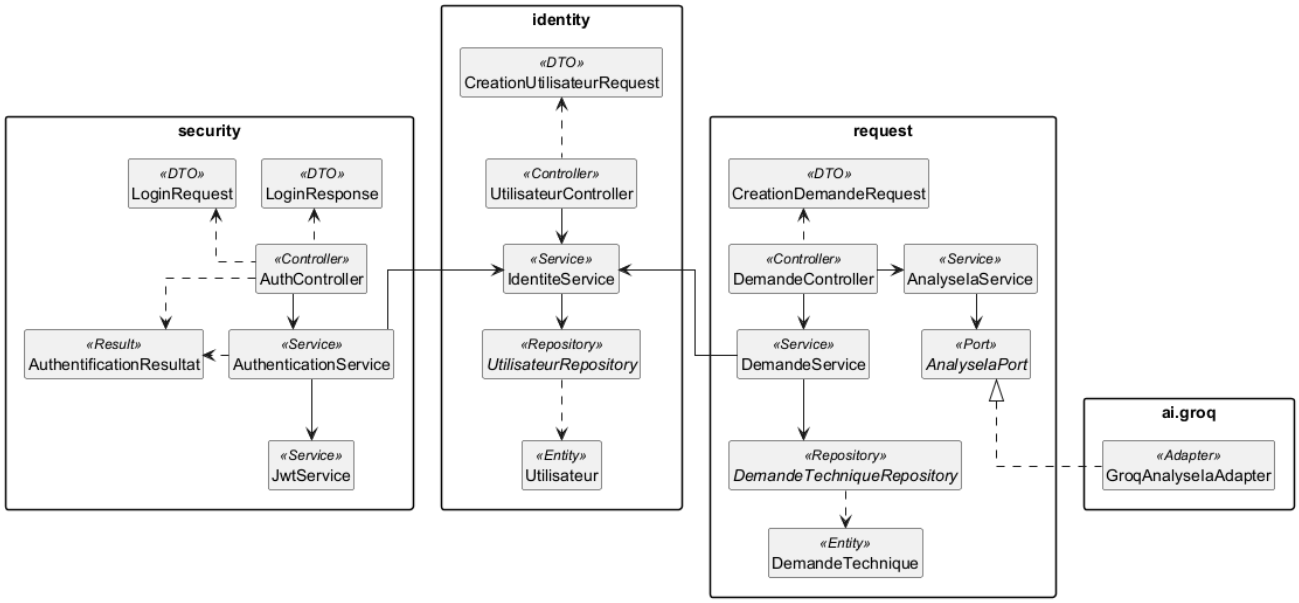


FIGURE 5 – Organisation des responsabilités dans la conception backend

Le diagramme précise que les contrôleurs portent la frontière HTTP, que les services orchestrent les cas d'utilisation et que le domaine protège les invariants. Les repositories isolent la persistance, tandis que le port et son adaptateur séparent l'intégration Groq du cœur applicatif.

2.3.3 Sécurité de la solution

L'**authentification** vérifie l'identité et l'état actif du compte avant l'émission d'un JSON Web Token (JWT) de durée limitée, transmis avec les requêtes protégées (Jones, Bradley et Sakimura 2015).

L'**autorisation** repose sur le contrôle d'accès basé sur les rôles (RBAC), complété par des contrôles contextuels liés à l'affectation, à l'état de la demande et à l'interdiction d'auto-désactivation de l'Administrateur.

Les **mots de passe** sont hachés avec BCrypt et les secrets proviennent d'une configuration externe. La conservation temporaire de la session dans le navigateur facilite le rechargement, mais ne vaut pas autorisation : le backend vérifie le JWT, l'état actif, les rôles courants et l'action demandée, et reste l'autorité.

2.3.4 Assistance intelligente

Le service analyse le titre et la description de la demande afin de proposer une catégorie et une priorité. Cette proposition reste soumise à une validation humaine, et la qualification manuelle demeure disponible en cas d'indisponibilité externe.

2.4 Conception de la base de données

La base relationnelle doit préserver l'identité des utilisateurs, le contenu et le cycle des demandes, puis la trace des événements. Le dictionnaire précise d'abord les principales données et leurs contraintes ; le modèle d'entités montre ensuite leurs relations.

TABLEAU 3 – Dictionnaire de données synthétique

Entité	Attribut	Type	Contrainte et signification
Utilisateur	id	Long / BIGINT	Clé primaire, séquence.
Utilisateur	email	String / VARCHAR	Obligatoire, normalisé et unique.
Utilisateur	actif	boolean / BOOLEAN	Obligatoire ; compte autorisé ou désactivé.
Utilisateur	roles	Set<Role>	Association ; plusieurs rôles possibles.
Client	id	Long / BIGINT	Clé primaire, séquence.
Client	nom, email, telephone	String / VARCHAR	Trois informations obligatoires ; email non unique.
DemandeTechnique	reference	String / VARCHAR	Obligatoire, unique et immuable ; référence métier.
DemandeTechnique	titre	String / VARCHAR	Obligatoire ; intitulé de la demande.
DemandeTechnique	description	String / TEXT	Obligatoire ; détail de la demande.
DemandeTechnique	categorie	Categorie / VARCHAR	Obligatoire ; valeur de la liste retenue.
DemandeTechnique	priorite	Priorite / VARCHAR	Obligatoire ; BASSE à CRITIQUE.
DemandeTechnique	statut	StatutDemande / VARCHAR	Obligatoire ; état du cycle de vie.
DemandeTechnique	agentAffecte	Utilisateur / FK	Optionnel ; au plus un Agent affecté.
DemandeTechnique	dates métier	Instant / timestamptz	Création et modification obligatoires ; résolution, clôture et annulation optionnelles.
HistoriqueDemande	typeEvenement	String / VARCHAR	Obligatoire ; nature du changement.
HistoriqueDemande	ancienne/nouvelle valeur	String / TEXT	Optionnelles ; valeurs avant et après.
HistoriqueDemande	demande, auteur	FK	Obligatoires ; origine et responsable de l'événement.

Le dictionnaire constitue la synthèse principale des attributs, types et contraintes. La figure 6 montre ensuite leurs relations et cardinalités.

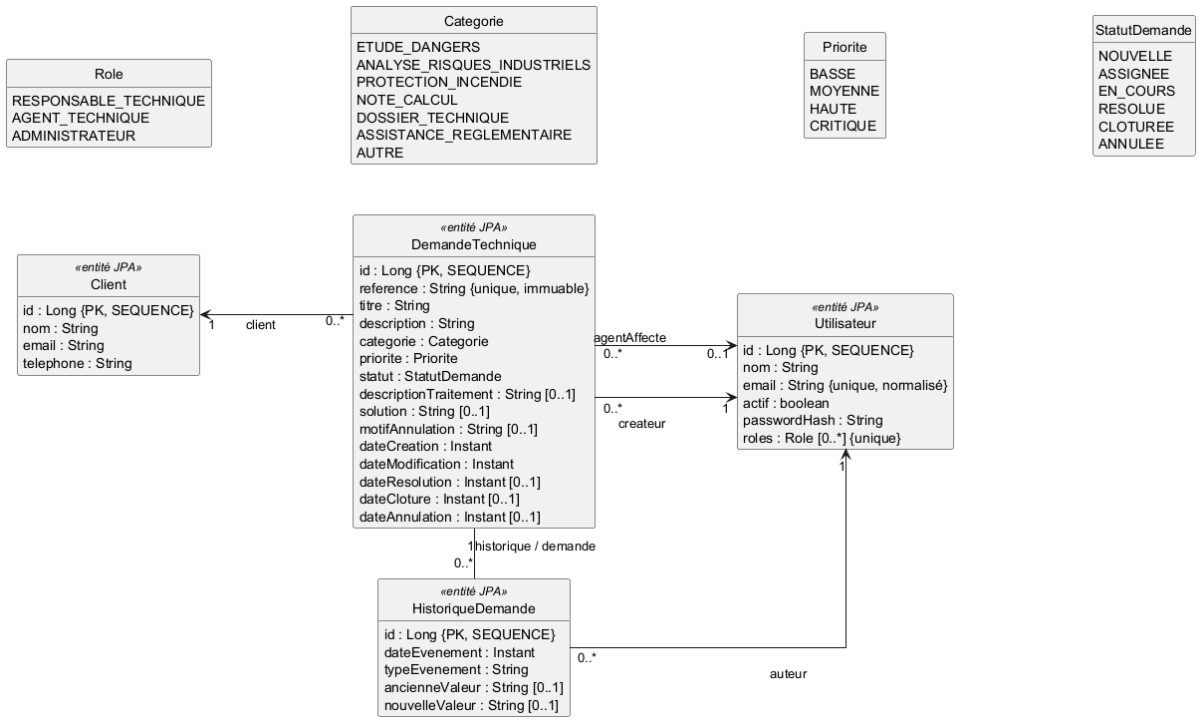


FIGURE 6 – Entités persistantes et cardinalités du modèle de données

`DemandeTechnique` constitue l'entité centrale : elle relie le Client, le créateur et, lorsqu'il existe, l'Agent affecté. `Utilisateur` porte l'identité et les rôles des acteurs internes, tandis que `Client` conserve les informations de la personne concernée par la demande.

`HistoriqueDemande` assure la traçabilité des changements et de leur auteur. Les instants sont stockés en `TIMESTAMP WITH TIME ZONE`, en cohérence avec le type `Instant` côté Java (PostgreSQL Global Development Group 2026). Les énumérations contrôlent les rôles, catégories, priorités et statuts autorisés sans devenir des entités indépendantes.

Conclusion

Ce chapitre a traduit les besoins en conception fonctionnelle, en architecture et en modèle de données. Il a également précisé la protection des accès et l'intégration d'une assistance IA soumise à validation humaine. Le chapitre suivant présente la réalisation de cette conception et les éléments qui permettent de la valider.

Chapitre 3

Réalisation et validation

Introduction

Ce chapitre présente la concrétisation de la conception décrite précédemment. Il expose l'environnement technique, les principales fonctionnalités réalisées, la protection des accès, l'assistance intelligente et les validations obtenues, avant d'en préciser les difficultés, les limites et les perspectives.

3.1 Environnement de réalisation et technologies utilisées

La solution s'organise autour d'une interface web Angular, d'un backend Spring Boot qui porte les règles métier, d'une base PostgreSQL et d'un service Groq externe. Le tableau 4 présente les versions pertinentes du projet ; les dépendances prises en charge par Spring Boot sont indiquées sans détailler leur résolution interne.

TABLEAU 4 – Technologies et versions du projet

Technologie	Version du projet	Rôle dans le projet
Java	21	Langage et cible d'exécution du backend.
Spring Boot et Spring Security	Spring Boot 4.1.1 ; Spring Security géré par Spring Boot	API métier, authentification et autorisation.
JPA, Hibernate et Flyway	Gérés par Spring Boot	Persistance relationnelle et migration du schéma.
PostgreSQL	Version serveur non spécifiée	Stockage des données métier.
Angular et Angular Material	Angular 22.2.1 ; Angular Material 22.2.2, versions verrouillées	Application web, navigation, formulaires et composants d'interface.
TypeScript	6.0.3 verrouillée	Développement typé de l'interface.
npm et Vitest	npm 11.19.0 déclarée ; Vitest 5.0.3 verrouillée	Gestion des dépendances, construction et tests frontend.
Groq	Modèle `qwen/qwen3.8-27b` configuré	Proposition de catégorie et de priorité.

Cette organisation maintient les règles métier et les contrôles de sécurité côté serveur. L'interface reste chargée des interactions, tandis que PostgreSQL et Groq sont accessibles à travers des frontières dédiées.

3.2 Réalisation des fonctionnalités principales

Les fonctionnalités ont été réalisées autour du cycle de vie d'une demande. Les actions proposées dans l'interface dépendent du rôle et de l'état courant, mais les règles correspondantes restent également contrôlées par le backend.

Création et qualification. Le Responsable technique renseigne le titre et la description, puis choisit un client existant ou saisit les informations minimales d'un nouveau client. Il valide ensuite la catégorie et la priorité ; l'assistance IA peut proposer ces deux valeurs, sans les imposer.

Le formulaire réunit les informations nécessaires à l'enregistrement sans mélanger la qualification et l'affectation. La demande n'est créée qu'après la validation des données finales par le Responsable technique.

Suivi et affectation. La liste permet de rechercher et filtrer les demandes, de les trier et de parcourir les résultats paginés par le serveur. Le Responsable technique peut ouvrir une demande

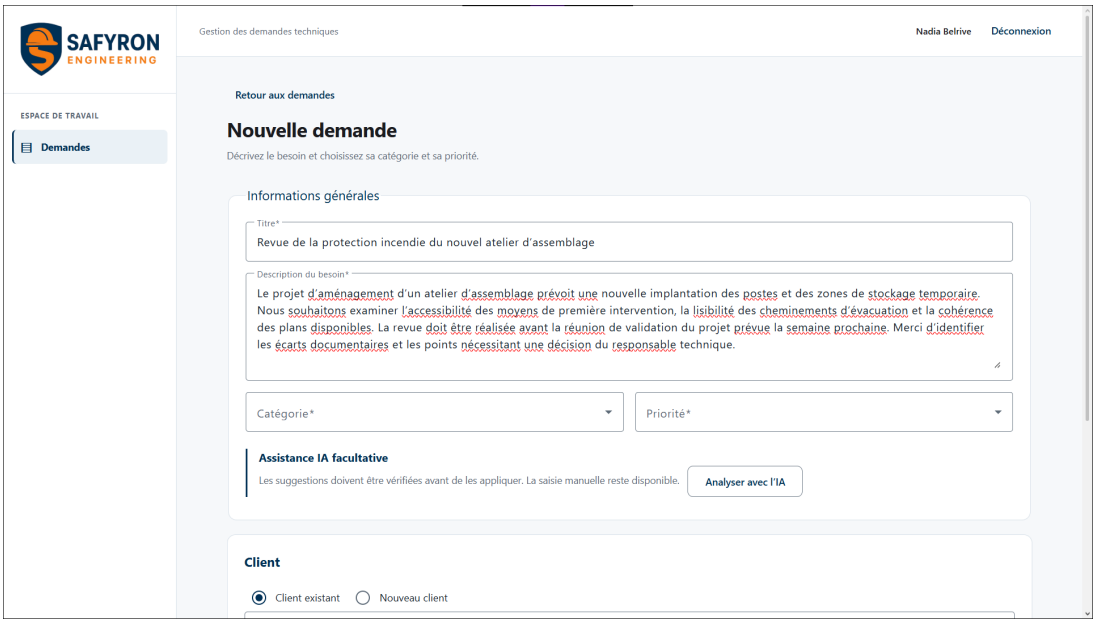


FIGURE 7 – Formulaire de création et de qualification d'une demande

puis l'affecter ou la réaffecter ; l'Agent dispose d'une visibilité adaptée aux demandes qu'il doit traiter.

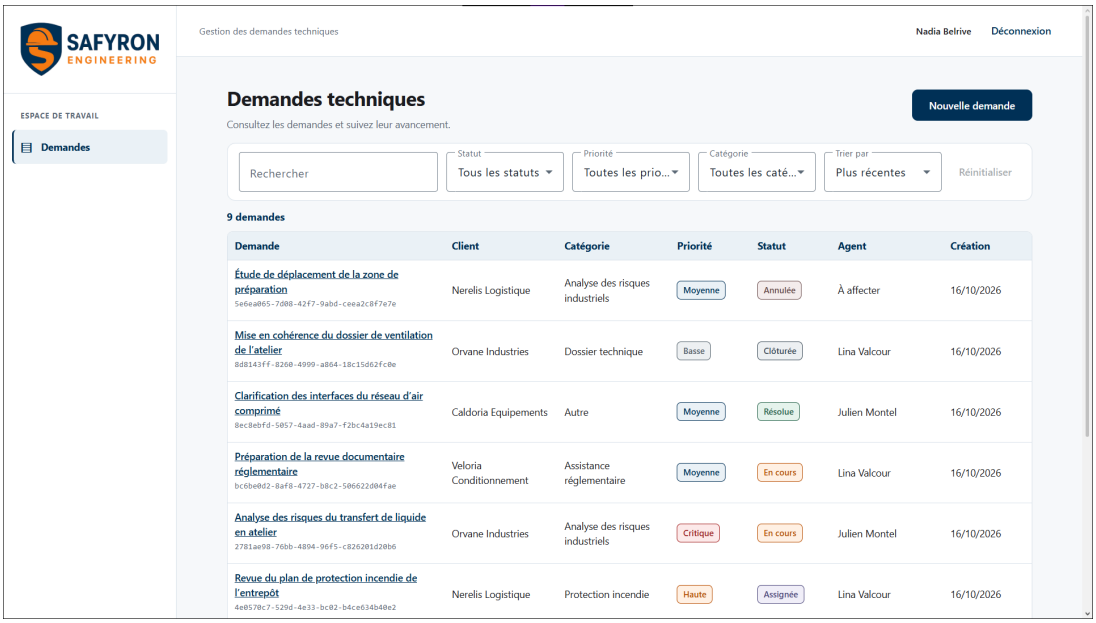


FIGURE 8 – Suivi des demandes avec filtres, tri et pagination

Cette vue donne une lecture synthétique du portefeuille de demandes et de leur état. Elle sert de point d'entrée vers les actions autorisées sur chaque dossier.

Traitement et résolution. Une fois affectée, la demande est prise en charge par l'Agent technique. Celui-ci démarre le traitement, documente progressivement son intervention, saisit la solution retenue puis déclare la demande résolue.



FIGURE 9 – Traitement d'une demande affectée

Le détail rassemble le contexte, l'affectation et la zone de traitement. Il permet à l'Agent de documenter la solution sans lui donner les décisions réservées au Responsable technique.

Examen et clôture. Le Responsable technique examine la résolution déclarée. Il peut clôturer la demande lorsqu'elle est satisfaisante ou la refuser ; dans ce dernier cas, la demande revient en traitement et la décision reste tracée.

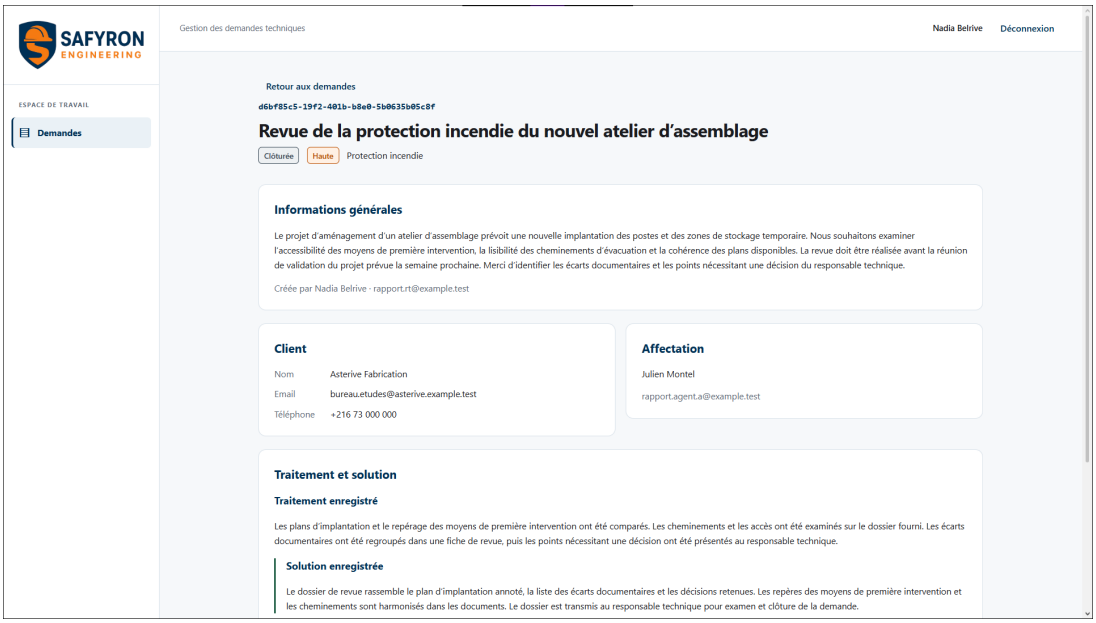


FIGURE 10 – Clôture après examen de la résolution

La clôture termine le cycle après une décision humaine explicite. L'interface rend le statut et la solution visibles, tandis que le backend vérifie que la transition demandée reste autorisée.

3.3 Administration et sécurité

L'administration permet de créer un utilisateur, de modifier son identité, de gérer son état actif et de remplacer ses rôles métier. Le rôle Administrateur est séparé des rôles de Responsable technique et d'Agent technique : ces derniers sont attribués comme rôles métier, tandis que l'administration reste une responsabilité distincte.

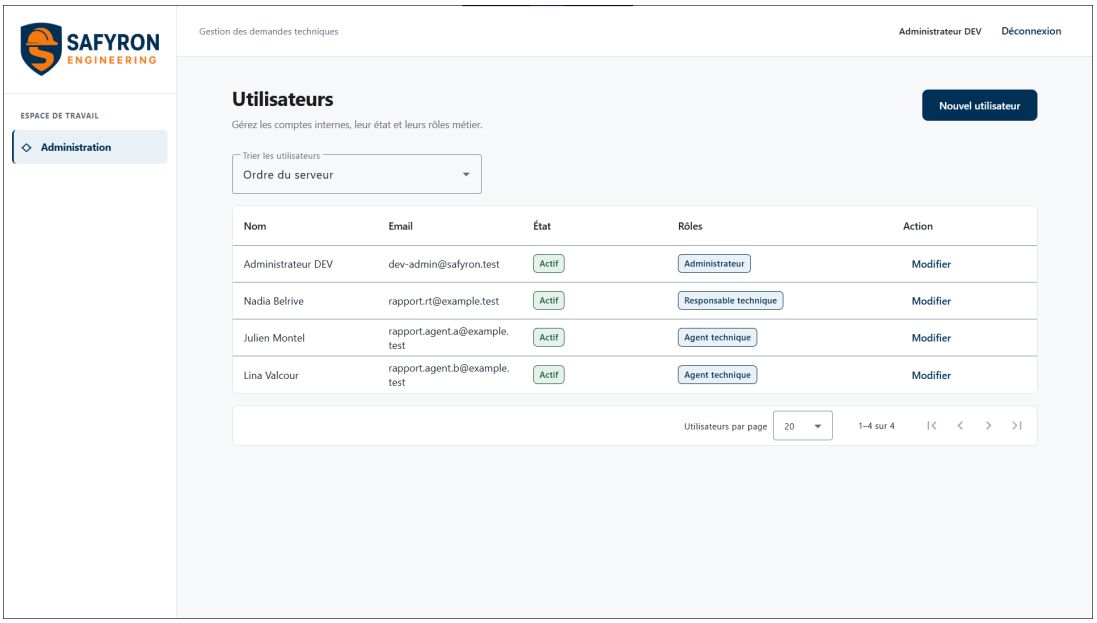


FIGURE 11 – Gestion des utilisateurs internes

La vue présente l'état et les rôles des comptes sans exposer les détails de sécurité. Elle permet d'accéder aux opérations d'administration tout en retirant l'action de désactivation pour le compte courant.

L'authentification repose sur un JSON Web Token (JWT) transmis dans l'en-tête des requêtes. L'API est sans session serveur : le token établit l'identité annoncée, puis le backend relit le compte, son état actif et ses rôles actuels pour chaque requête protégée.

L'autorisation associe les rôles métier à des contrôles contextuels. Par exemple, un Agent ne traite que les demandes qui lui sont affectées et une transition dépend du statut courant. Les mots de passe sont hachés avec BCrypt ; les secrets JWT, le coût BCrypt et les origines autorisées par la politique CORS proviennent de la configuration externe.

Angular conserve temporairement la session dans `sessionStorage` afin de la restaurer après un rechargement. Cette conservation ne rend pas le navigateur autoritatif : un compte désactivé ou un rôle retiré perd ses droits dès la requête protégée suivante, et une réponse 401 invalide la session locale.

3.4 Assistance IA et validation humaine

Le Responsable technique déclenche volontairement l'analyse depuis le formulaire de création. Angular transmet uniquement le titre et la description au backend. Le service passe par le port d'analyse défini lors de la conception, puis l'adaptateur Groq appelle le modèle configuré et contrôle que la réponse structurée contient exactement une catégorie et une priorité reconnues.

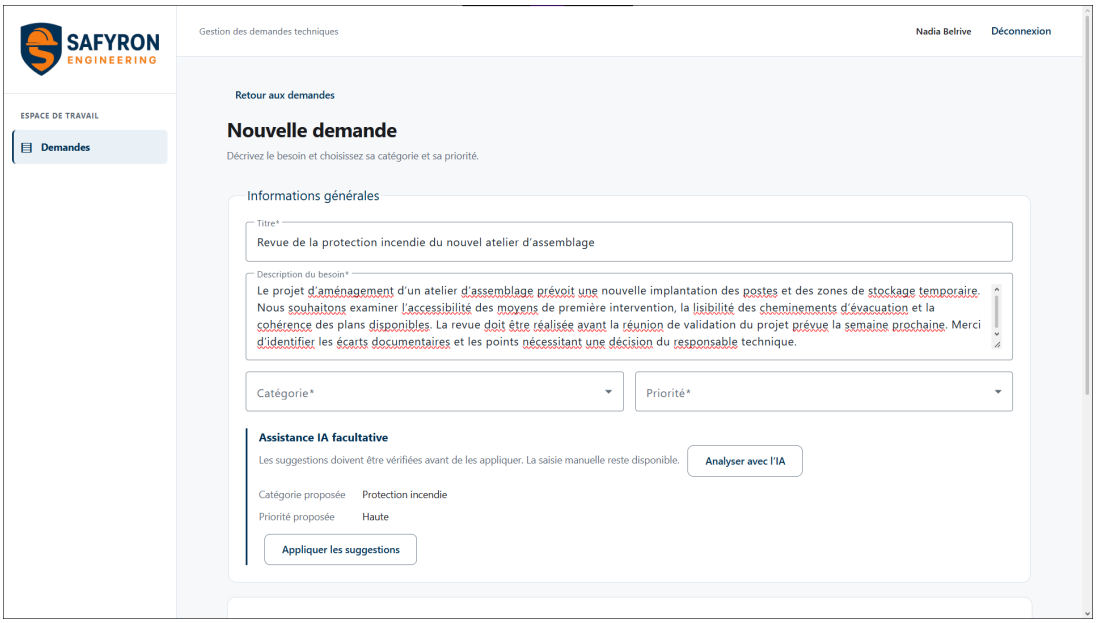


FIGURE 12 – Assistance IA lors de la qualification

Les valeurs retournées restent des propositions. Elles ne sont appliquées qu'après une action humaine explicite et demeurent modifiables avant la création. Une modification du titre ou de la description invalide la suggestion et impose une nouvelle analyse volontaire.

L'indisponibilité de Groq constitue une contrainte d'intégration prévue et testée. Une clé absente, une réponse invalide ou une indisponibilité externe rend seulement l'analyse IA indisponible ; la saisie manuelle de la catégorie et de la priorité reste accessible.

Le benchmark exploratoire utilise le jeu P01–P24, soit 24 cas dont la catégorie et la priorité proposées sont comparées aux valeurs attendues puis relues humainement. Les 24 sorties sont valides ; la catégorie est correcte dans 23 cas, la priorité dans 22 cas et la correspondance exacte dans 22 cas. Ce petit corpus ne représente ni la diversité future des demandes ni une mesure générale de performance ; les résultats restent liés à ce jeu, à cette configuration et à cette évaluation humaine.

3.5 Tests et validation

La validation combine des tests automatisés, des vérifications d'intégration réelle et des parcours humains. Le tableau 5 sépare ces niveaux afin de ne pas assimiler les tests H2 à une validation PostgreSQL ou à un appel Groq réel. Les workflows automatisés ont été réexécutés pour cette vérification ; les lignes relatives à PostgreSQL réel, à Groq réel et aux parcours navigateur reprennent des validations effectivement réalisées et documentées auparavant, sans les présenter comme réexécutées pendant cette passe.

TABLEAU 5 – Résultats de validation par niveau

Niveau	Méthode	Résultat vérifié
Backend automatisé	Maven `clean test` et `clean verify`	270 tests réussis ; aucun échec, erreur ou test ignoré.
Frontend automatisé	Build Angular et tests Vitest	Build réussi ; 207 tests réussis dans 16 fichiers.
PostgreSQL réel	Démarrage et appels HTTP locaux	Connexion réussie, migration Flyway appliquée, validation Hibernate et workflow API vérifiés.
IA hors ligne	H2 et stub HTTP local	Comportements IA testés sans appel réel à Groq.
Intégration IA réelle	Parcours Angular vers Spring Boot puis Groq	Proposition reçue et appliquée seulement après action humaine ; scénario sans clé vérifié en 503.
Validation humaine	Parcours navigateur et revue responsive	Connexion, création, affectation, traitement, résolution, refus, clôture, administration, IA et UX vérifiés.

Preuves textuelles d'exécution. Les sorties complètes des commandes réexécutées sont conservées dans `report/figures/results/`. Les extraits suivants reprennent directement leurs résumés, sans reconstitution graphique.

Backend — Maven `clean test` et `clean verify`

```
Tests run: 270, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Frontend — build Angular et Vitest

```
Application bundle generation complete.
Test Files 16 passed (16)
Tests 207 passed (207)
```

Le tableau 6 rapproche ensuite les besoins non fonctionnels du chapitre 2 des éléments réellement disponibles. Il distingue les résultats de tests, les vérifications humaines et les indicateurs issus de la conception.

TABLEAU 6 – Traçabilité des besoins non fonctionnels et des validations

Besoin	Élément de validation ou indicateur disponible
Sécurité	Tests backend portant sur JWT, RBAC, CORS, la relecture de l'identité et les contrôles métier contextuels ; ces résultats ne constituent pas un audit de sécurité.
Cohérence	Tests du domaine, du workflow, de persistance et des API, complétés par les parcours fonctionnels vérifiés.
Traçabilité	Historique des changements intégré au modèle et scénarios API associés, avec consultation vérifiée dans le parcours de demande.
Ergonomie	Vérifications humaines dans le navigateur, revue responsive et captures réelles ; elles ne remplacent pas une étude UX exhaustive.
Continuité de l'IA	Tests hors ligne avec stub, scénario sans clé renvoyant 503 et maintien vérifié de la qualification manuelle.
Maintenabilité	Indicateurs de conception : modularité, séparation des responsabilités, tests automatisés et isolation de Groq par un port et un adaptateur ; aucune mesure scientifique de maintenabilité n'est revendiquée.

Ces éléments confirment le périmètre fonctionnel et technique effectivement vérifié. Ils ne constituent ni un test de performance ou de charge, ni un test d'intrusion, ni une qualification exhaustive de sécurité ou de production.

3.6 Difficultés rencontrées, limites et perspectives

Difficulté rencontrée et corrigée. La validation navigateur a révélé qu'un administrateur pouvait tenter de désactiver son propre compte. Le backend refuse désormais cette opération avec une réponse HTTP 409, l'interface masque l'action concernée et conserve un message compréhensible. Le retest humain a confirmé le refus de l'auto-désactivation et le maintien de la désactivation des autres comptes.

Contrainte technique prévue. L'assistance dépend d'un service Groq externe dont la disponibilité et la configuration ne sont pas garanties. La conception isole cette dépendance et la rend non bloquante. Le scénario sans clé a été testé : seule l'analyse renvoie une indisponibilité, tandis que la qualification manuelle demeure opérationnelle.

Limites actuelles. La solution actuelle reste réservée aux utilisateurs internes et ne propose pas de portail client. Le benchmark IA repose sur un corpus exploratoire de 24 cas. Aucun test de charge ni validation de production n'a été réalisé. Enfin, le JWT est conservé dans `sessionStorage` pour permettre la restauration de session, avec les risques propres à un stockage accessible au JavaScript exécuté dans le navigateur.

Perspectives. Les évolutions suivantes ne font pas partie de la solution actuelle et nécessiteraient chacune une analyse et une validation spécifiques.

L'assistance IA pourrait être étudiée au-delà de la proposition de catégorie et de priorité, par exemple pour extraire des informations utiles, synthétiser le besoin ou attirer l'attention sur des éléments importants pour le traitement. Toute nouvelle proposition resterait soumise à une validation humaine. Son évaluation devrait également reposer sur un corpus plus large et des mesures répétées.

Un espace de discussion associé à chaque demande pourrait faciliter les échanges entre les acteurs internes et conserver les informations opérationnelles avec le dossier concerné. Cette évolution demanderait de définir les droits d'accès, l'historisation, les notifications et les règles de confidentialité, sans préjuger de la technologie de communication.

L'ajout de pièces jointes permettrait d'associer aux demandes et à leur traitement des documents, des images ou certains fichiers techniques. Une conception future devrait encadrer les types autorisés, la taille maximale, le stockage, la sécurité et le contrôle des fichiers.

Enfin, un espace propre au Client pourrait permettre la soumission d'une demande, la consultation de son état et l'accès aux échanges ou documents autorisés. Cette évolution modifierait le périmètre actuel : elle imposerait une authentification client, de nouvelles règles d'autorisation et de protection des données ainsi que de nouveaux cas d'utilisation. Le Client demeure, dans la solution réalisée, une entité métier et non un utilisateur. Une meilleure observabilité resterait également souhaitable pour faciliter le suivi des erreurs et des dépendances externes.

Conclusion

Ce chapitre a montré la réalisation des choix de conception à travers le parcours des demandes, l'administration, la sécurité et l'assistance IA sous contrôle humain. Les validations automatisées, réelles et humaines confirment le périmètre vérifié, tout en laissant apparaître des limites clairement identifiées. Ces éléments permettent d'aborder la conclusion générale avec une vision équilibrée des résultats obtenus.

Conclusion générale

Le projet a abouti à une application couvrant l'enregistrement, la qualification, l'affectation, le traitement, la résolution et la clôture des demandes techniques. La séparation entre interface Angular, API Spring Boot et persistance PostgreSQL fournit une base lisible et évolutive. La sécurité combine authentification JWT, rôles métier et contrôles contextuels.

L'assistance IA facultative via Groq propose une catégorie et une priorité, mais la décision finale reste modifiable et validée par le Responsable technique. Les tests automatisés, les vérifications d'intégration réelle et les validations humaines confirment le fonctionnement du périmètre retenu pour le stage dans les conditions vérifiées.

Ce stage a permis de mettre en pratique une démarche complète de génie logiciel, depuis la compréhension du besoin et sa modélisation UML jusqu'à la conception, la réalisation, les tests et la validation. Il a également renforcé la distinction entre les besoins fonctionnels et les choix techniques, ainsi que la nécessité de justifier une architecture par les responsabilités qu'elle doit séparer. La réalisation individuelle du projet a demandé de l'autonomie et une progression contrôlée, notamment pour articuler Angular, Spring Boot et PostgreSQL, mettre en œuvre les contrôles JWT et RBAC, puis intégrer un service IA externe sans lui confier la décision métier.

Les limites concernent le périmètre interne, l'absence de portail client et le caractère exploratoire du benchmark IA. À titre de perspectives, des travaux ultérieurs pourraient approfondir l'assistance IA, faciliter la communication entre les acteurs, prendre en charge des pièces jointes et étudier un futur espace client. Ils pourraient aussi renforcer l'observabilité et élargir l'évaluation. Ces fonctions ne font pas partie de la solution actuelle et exigeraient une analyse du besoin, une conception et des validations supplémentaires.

Références

Cockburn, Alistair (4 sept. 2005). *Hexagonal Architecture*. HaT Technical Report 2005.02. url : https://alistair.cockburn.us/hexagonal-architecture/ (visité le 27/10/2026).

Fowler, Martin (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional. isbn : 9780321127426.

Gamma, Erich et al. (1994). *Design Patterns : Elements of Reusable Object-Oriented Software*. 1re éd. Addison-Wesley Professional. isbn : 9780201633610.

Jones, Michael, John Bradley et Nat Sakimura (mai 2015). *JSON Web Token (JWT)*. url : https://www.rfc-editor.org/rfc/rfc7519.html (visité le 11/10/2026).

Martin, Robert C. (2017). *Clean Architecture : A Craftsman's Guide to Software Structure and Design*. 1re éd. Pearson. isbn : 9780134494166.

PostgreSQL Global Development Group (2026). *Date/Time Types*. url : https://www.postgresql.org/docs/current/datatype-datetime.html (visité le 11/10/2026).